APPENDIX 1 — GITHUB MARKDOWN (MD) LANGUAGE FAMILIARIZATION

AIM

To familiarize with GitHub Markdown (.md) language — its various formatting tags, syntax, and rendered output — and to create a structured README.md file for an IoT lab repository.

INTRODUCTION

Markdown is a lightweight markup language (John Gruber, 2004) that converts plain text to HTML. GitHub uses GitHub Flavored Markdown (GFM) — an extended version adding tables, task lists, strikethrough, fenced code blocks, and emoji.

Property	Detail
File Extension	.md or .markdown
GitHub Flavor	GFM — superset of CommonMark
Used For	README files, documentation, wikis, issues, pull requests
Conversion	Markdown → HTML (auto-rendered by GitHub)

MARKDOWN TAGS AND THEIR OUTPUT

1. Headings

Prefix with # signs. Number of # = heading level H1 to H6.

Markdown Syntax	Rendered Output (GitHub)	Notes
`# Heading 1`	`[ Large bold title – H1 ]`	Page/section title
`## Heading 2`	`[ Slightly smaller – H2 ]`	Main section
`### Heading 3`	`[ Medium – H3 ]`	Sub-section
`#### Heading 4`	`[ Small – H4 ]`	Minor heading

2. Text Emphasis

Markdown Syntax	Rendered Output (GitHub)	Notes
`**Bold Text**`	`Bold Text`	Two asterisks around text
`__Bold Text__`	`Bold Text`	Two underscores — same
`*Italic Text*`	`Italic Text`	Single asterisk
`_Italic Text_`	`Italic Text`	Single underscore — same
`***Bold & Italic***`	`Bold and Italic`	Three asterisks
`~~Strikethrough~~`	`Strikethrough`	Two tildes — GFM only
`` `inline code` ``	`inline code (monospace)`	Backtick for terms/commands

3. Lists

Markdown Syntax	Rendered Output (GitHub)	Notes
`- Item A - Item B - Item C`	• Item A • Item B • Item C	Unordered with dash
`* Item A * Item B`	• Item A • Item B	Same using asterisk
`1. First 2. Second 3. Third`	1. First 2. Second 3. Third	Ordered list
`- Parent - Child - Child`	• Parent ○ Child ○ Child	Nested (2-space indent)

4. Links and Images

Markdown Syntax	Rendered Output (GitHub)	Notes
`[Link Text](https://url.com)`	Link Text (clickable)	Inline hyperlink
`![Alt text](image.png)`	[Image rendered inline]	! prefix = image
`<https://url.com>`	https://url.com (auto-link)	Angle bracket auto-link

5. Blockquotes

Markdown Syntax	Rendered Output (GitHub)	Notes
`> This is a quote`	| This is a quote	Single >
`> Line 1 > Line 2`	| Line 1 | Line 2	Multi-line
`> Outer >> Nested`	| Outer | | Nested	Nested blockquote

6. Code Blocks

Markdown Syntax	Rendered Output (GitHub)	Notes
`` `analogRead(34)` ``	analogRead(34) (mono)	Inline — single backtick
```` ``` code here ``` ````	[Monospace block]	Fenced code block
```` ```cpp void loop(){} ``` ````	[C++ syntax highlighted]	Language tag = highlighting
```` ```python print("Hi") ``` ````	[Python highlighted]	python tag
```` ```bash npm install ``` ````	[Shell highlighted]	bash tag

7. Tables (GFM)

Markdown Syntax	Rendered Output (GitHub)	Notes
`| Col1 | Col2 | |------|------| | A | B |`	[Table with borders]	Basic table
`|:Left|:Center:|Right:|`	[Aligned columns]	: controls alignment

8. Horizontal Rule, Task Lists, Emoji

Markdown Syntax	Rendered Output (GitHub)	Notes
`---`	——————————— (full-width line)	Three or more hyphens
`***`	——————————— (same)	Three asterisks
`- [ ] Unchecked task`	☐ Unchecked task	GFM checkbox (empty)
`- [x] Completed task`	☑ Completed task	GFM checkbox (checked)
`:smile:`	😄	Colon-wrapped emoji name
`:rocket:`	🚀	Common in README badges
`:white_check_mark:`	✅	Green tick

9. HTML in Markdown

Markdown Syntax	Rendered Output (GitHub)	Notes
` `	[Line break]	Explicit line break
`<b>Bold</b>`	**Bold**	HTML bold tag
`<center>Text</center>`	Text (centered)	Center alignment
`<details><summary>Click</summary> Hidden </details>`	[Collapsible section]	Expandable block

10. Special GFM Features

Feature	Usage
Mermaid Diagrams	```mermaid block — renders flowcharts, sequence diagrams on GitHub
Math (KaTeX)	$E=mc^2$ inline or $$formula$$ block renders LaTeX math
Alerts (GFM 2023)	> [!NOTE] / [!TIP] / [!WARNING] render coloured callout boxes
Issue linking	#123 auto-links to Issue/PR number 123 in the same repo
@mentions	@username notifies that GitHub user

PRACTICAL: README.md FOR IoT LAB REPOSITORY

The following README.md demonstrates all the major Markdown elements covered above:

```
# PEMRT525 — IoT & Applications
> MR 2K24 S5 | Mechatronics | JECC

## About This Repository
This repo contains **lab experiments** and *project files*.

---

## Experiments List

| No. | Experiment | Status |
|-----|-----------|--------|
| 1   | GitHub Markdown         | Done |
```

```
| 2   | Relay control via Blynk | Done |
| 3   | EM18 RFID Basic         | Done |

## How to Use
1. Clone the repository
2. Open the Arduino sketch
3. Update Wi-Fi credentials
4. Upload to ESP32

## File Structure
```
IoT-Lab/
├── Exp1_Relay_Blynk/
│   └── relay_blynk.ino
└── README.md
```


## Tasks
- [x] Push relay experiment
- [ ] Add GPS experiment

> **Tip:** Always set Serial Monitor baud to **115200**.
```

PROCEDURE

1. Open VS Code and create a new file named README.md.
2. Type the practical README content shown above.
3. Press Ctrl+Shift+V in VS Code to preview the rendered Markdown in a side panel.
4. Observe how # creates headings, ** creates bold, - creates bullet lists, and ``` creates code blocks.
5. Create a GitHub account at https://github.com if you do not have one.
6. Click "+" → New repository → give a name → Initialize with README.
7. Click the README.md pencil (Edit) icon on GitHub. Paste your content.
8. Use the "Preview" tab to see the rendered output before committing.
9. Click "Commit changes" to save.
10. Observe the formatted README on your repository home page.
11. Experiment by adding a table, task list, and fenced code block with ```cpp highlighting.

OBSERVATION TABLE

Sl.	MD Tag	Rendered Output Observed	Remarks
1	`# H1`		
2	`**bold**`		
3	`*italic*`		
4	`- list`		
5	`1. ordered`		
6	`> quote`		

7	```code```		
8	| table |		
9	- [x] task		
10	---		

RESULT

RESULT

GitHub Markdown (.md) language was studied and practised. Major formatting tags including headings (H1–H6), text emphasis, ordered and unordered lists, links, images, blockquotes, fenced code blocks with syntax highlighting, tables, task lists, horizontal rules, and emoji were demonstrated. A complete README.md file for the IoT lab GitHub repository was created and verified using the GitHub web editor preview.

APPENDIX 2 — RELAY CONTROL VIA BLYNK IoT (ESP32 + SL100)

AIM

To control a relay using the Blynk IoT platform through an ESP32 microcontroller, with an SL100 NPN transistor as the relay driver, allowing ON/OFF control of a load from a mobile application over the internet.

COMPONENTS REQUIRED

Sl.	Component	Specification	Qty	Purpose
1	Microcontroller	ESP32 Dev Board	1	WiFi + Blynk control
2	Relay	5V SPDT Relay	1	Switch load circuit
3	Transistor	SL100 NPN	1	Drive relay coil
4	Resistor	1 kΩ, 1/4 W	1	Base current limiting
5	Diode	1N4007	1	Flyback protection across relay coil
6	LED + 470 Ω	Any colour	1	Load indicator (optional)
7	Breadboard + Jumpers	—	1 set	Connections
8	Smartphone	Android / iOS with Blynk	1	Remote control

CIRCUIT DIAGRAM

```
ESP32 GPIO2 ──[1kΩ]──→ Base of SL100
                       Emitter → GND
                       Collector → Relay Coil (−)
5V ──────────────────→ Relay Coil (+)

Relay COM → Load Supply
Relay NO  → Load
Load other terminal → Supply return

GPIO2 HIGH → SL100 ON → Relay energised → Load ON
GPIO2 LOW  → SL100 OFF → Relay de-energised → Load OFF
```

Figure A2.1: ESP32 GPIO2 → 1kΩ → SL100 base → relay coil → load

Connection	From	To	Notes
1kΩ resistor	ESP32 GPIO2	SL100 Base	Limits base current
SL100 Emitter	—	GND	Common ground
SL100 Collector	—	Relay Coil (−)	Drives relay
Relay Coil (+)	—	5V (VIN)	Relay supply
1N4007 diode	Relay Coil (−)	Relay Coil (+)	Cathode to + : flyback

			protection
Relay NO	—	Load	Normally Open contact

THEORY

The SL100 is an NPN silicon transistor. A small base current (from GPIO2 through 1kΩ) drives a larger collector current through the relay coil, energising the relay. When GPIO2 is LOW, the transistor is cut off and the relay is de-energised.

Blynk IoT Platform:

Virtual Pin	Widget	Action
V1	Button (Switch mode)	Sends 1 (ON) or 0 (OFF) to ESP32 via BLYNK_WRITE(V1) handler

The 1N4007 diode across the relay coil provides flyback protection — when the coil is de-energised, the collapsing magnetic field generates a reverse voltage spike that would damage the transistor without the diode.

PROGRAM CODE (ESP32 — Arduino IDE)

```
#define BLYNK_TEMPLATE_ID   "TMPLxxxxxxxxx"
#define BLYNK_TEMPLATE_NAME "Relay Control"
#define BLYNK_AUTH_TOKEN    "YourAuthTokenHere"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>

#define RELAY_PIN  2   // GPIO2 drives SL100 base via 1kΩ

char ssid[] = "Your_WiFi_SSID";
char pass[] = "Your_WiFi_Password";

BLYNK_WRITE(V1) {            // Blynk app Button on Virtual Pin V1
  int val = param.asInt();   // 1 = ON, 0 = OFF
  digitalWrite(RELAY_PIN, val);
  Serial.print("Relay: ");
  Serial.println(val ? "ON" : "OFF");
}

void setup() {
  Serial.begin(115200);
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, LOW);   // relay OFF at start
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
}

void loop() {
  Blynk.run();
```

```
}
```

BLYNK SETUP

1. Sign in at https://blynk.cloud and create a new Template (Hardware: ESP32, Connection: WiFi).
2. Add a Datastream: V1 (Integer, Min 0, Max 1).
3. In Web Dashboard, add a Button widget → assign to V1 → set to Switch mode.
4. Copy TEMPLATE_ID, TEMPLATE_NAME, and AUTH_TOKEN into the code.
5. In the Blynk mobile app, create a dashboard with a Button widget on V1.

PROCEDURE

1. Build the circuit: GPIO2 → 1kΩ → SL100 base; SL100 collector → relay coil (−); 5V → relay coil (+); 1N4007 across coil; relay NO → load.
2. Install Blynk library in Arduino IDE (Sketch → Manage Libraries → search "Blynk").
3. Update BLYNK_TEMPLATE_ID, BLYNK_AUTH_TOKEN, ssid, and pass in the code.
4. Upload the code. Open Serial Monitor at 115200 baud.
5. Confirm "Connected to Blynk" appears in Serial Monitor.
6. Open the Blynk app → press the Button widget.
7. Observe the relay clicking and the load switching ON and OFF.
8. Verify relay state matches the button state in the app.

RESULT

RESULT

The relay was successfully controlled through the Blynk IoT platform using an ESP32 and SL100 transistor driver. The BLYNK_WRITE(V1) handler received button state changes from the Blynk app over WiFi and toggled GPIO2 accordingly, driving the SL100 transistor which energised or de-energised the relay coil, switching the connected load ON and OFF remotely.

APPENDIX 3 — EM18 RFID READER BASIC INTERFACE (ESP32)

AIM

To interface the EM18 125 kHz RFID reader with an ESP32 microcontroller using UART serial communication and read and display the 12-character card UID on the Serial Monitor whenever a compatible card is presented.

COMPONENTS REQUIRED

Sl.	Component	Specification	Qty	Purpose
1	Microcontroller	ESP32 Dev Board	1	Read serial data
2	RFID Reader	EM18, 125 kHz	1	Read card UID
3	RFID Cards / Key Fobs	EM4100, 125 kHz	2+	Tags to scan
4	Resistor	1 kΩ, 1/4 W	1	Voltage divider upper
5	Resistor	2 kΩ, 1/4 W	1	Voltage divider lower
6	Breadboard + Jumpers	—	1 set	Connections

CIRCUIT DIAGRAM

IMPORTANT: EM18 TX is 5V TTL. Use 1kΩ/2kΩ voltage divider to bring signal to ~3.33 V. SEL must be connected to GND to enable UART output.

```
EM18 Module                    ESP32
VCC  → 5V (VIN)
GND  → GND
TX   → [1kΩ] → node → GPIO3 (RX0)
                 node → [2kΩ] → GND
SEL  → GND          (enable UART mode)
BEEP → [1kΩ] → LED → GND  (optional)

Voltage at node = 5 × 2/(1+2) = 3.33 V  (safe for ESP32)
```

Figure A3.1: EM18 TX → 1kΩ/2kΩ divider → GPIO3 (RX0)

THEORY

The EM18 reads passive EM4100 cards at 125 kHz. When a card is in range (~8–12 cm), it reads the 40-bit unique ID and transmits 12 ASCII bytes over UART at 9600 baud.

Bytes	Content	Description
0–1	"XX" (2 hex chars)	Version / Customer code byte
2–9	"XXXXXXXX" (8 hex chars)	Card unique ID (4 bytes)
10–11	"XX" (2 hex chars)	XOR checksum of all 5 data bytes

SEL pin: LOW (GND) = UART serial output; HIGH or floating = Wiegand output. Must be tied to GND for this experiment.

PROGRAM CODE (ESP32 — Arduino IDE)

```
/* EM18 RFID Basic – ESP32
   EM18 TX → 1kΩ/2kΩ divider → GPIO3 (RX0)
   EM18 SEL → GND (UART mode)
   Serial baud: 9600 */

int count = 0;
char card_no[12];

void setup()
{
   Serial.begin(9600);
}

void loop()
{
   if (Serial.available())
   {
      count = 0;
      while (Serial.available() && count < 12)
      {
         card_no[count] = Serial.read();
         count++;
         delay(5);
      }
      Serial.print(card_no);
   }
}
```

PROCEDURE

1. Build the voltage divider: 1kΩ from EM18 TX to a node; 2kΩ from that node to GND; node to GPIO3 (RX0).
2. Connect EM18 VCC to VIN (5V) and GND to GND. Connect SEL to GND.
3. Upload the code. Open Serial Monitor at 9600 baud.
4. Bring an EM4100 125 kHz card within 8–12 cm of the reader.
5. Observe the 12-character card data printed in the Serial Monitor.
6. Test with a second card and confirm a different ID is displayed.

RESULT

RESULT

The EM18 125 kHz RFID reader was successfully interfaced with the ESP32 using UART0 (GPIO3/RX0) through a 1kΩ/2kΩ voltage divider. The SEL pin was tied to GND to select UART

mode. When a 125 kHz card was brought near the reader, the 12-character card data was received and displayed on the Serial Monitor at 9600 baud.

APPENDIX 4 — EM18 RFID ACCESS CONTROL SYSTEM (ESP32)

AIM

To implement an RFID-based access control system using the EM18 reader and ESP32, where only pre-registered card UIDs are granted access (LED ON for 1 second), and unregistered cards are denied (LED blinks 3 times), with XOR checksum verification.

COMPONENTS REQUIRED

Sl.	Component	Specification	Qty	Purpose
1	Microcontroller	ESP32 Dev Board	1	Access control logic
2	RFID Reader	EM18, 125 kHz	1	Read card UID
3	RFID Cards	EM4100 125 kHz	2+	Authorised + unauthorised tags
4	LED	Any colour, 5mm	1	Access indicator
5	Resistor	470 Ω, 1/4 W	1	LED current limiting
6	Resistor	1 kΩ + 2 kΩ	1 each	Voltage divider for EM18 TX

CIRCUIT DIAGRAM

EM18 Pin	Connection	Notes
VCC	5V (VIN)	EM18 supply
GND	GND	Common ground
TX	1kΩ → node → GPIO16 (RX2)	2kΩ from node to GND; node to GPIO16
SEL	GND	UART mode select
LED Anode	GPIO2 → 470Ω → LED	Access indicator

Uses Serial2 (GPIO16/RX2) to leave Serial0 free for debug output.

THEORY

XOR Checksum Verification:

The EM18 sends 12 ASCII characters: 10 hex characters (5 data bytes) + 2 hex characters (XOR checksum). The checksum is XOR of all 5 data bytes. The checksumOk() function verifies integrity before accepting the card ID.

Access Control Logic:

Condition	LED Response	Serial Output
Checksum fails	No response	"Checksum error"
UID in allowedIDs[]	ON for 1 second	"ACCESS GRANTED"

UID not in allowedIDs[]	Blinks 3 times (200 ms each)	"ACCESS DENIED"

PROGRAM CODE (ESP32 — Arduino IDE)

```
#define RXD2     16
#define TXD2     17
#define LED_PIN   2

const byte   NUM_ALLOWED = 2;
const String allowedIDs[NUM_ALLOWED] = {"1A2B3C4D5E", "0011223344"};

bool isAllowed(const String &id) {
  for (byte i = 0; i < NUM_ALLOWED; i++)
    if (id == allowedIDs[i]) return true;
  return false;
}

byte hexVal(char c) {
  if (c >= "0" && c <= "9") return c - "0";
  if (c >= "A" && c <= "F") return c - "A" + 10;
  if (c >= "a" && c <= "f") return c - "a" + 10;
  return 0;
}

bool checksumOk(const char *buf) {
  byte x = 0;
  for (int i = 0; i < 10; i += 2)
    x ^= (hexVal(buf[i]) << 4) | hexVal(buf[i+1]);
  byte received = (hexVal(buf[10]) << 4) | hexVal(buf[11]);
  return x == received;
}

void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);
  Serial.println("RFID Access Control Ready");
}

void loop() {
  if (Serial2.available() >= 12) {
    char buf[13]; buf[12] = 0;
    for (int i = 0; i < 12; i++) {
      buf[i] = Serial2.read();
      delay(5);
    }
    String uid = String(buf).substring(0, 10);
    Serial.print("Card: "); Serial.println(uid);

    if (!checksumOk(buf)) {
```

```
      Serial.println("Checksum error");
      return;
    }
    if (isAllowed(uid)) {
      Serial.println("ACCESS GRANTED");
      digitalWrite(LED_PIN, HIGH);
      delay(1000);
      digitalWrite(LED_PIN, LOW);
    } else {
      Serial.println("ACCESS DENIED");
      for (int i = 0; i < 3; i++) {
        digitalWrite(LED_PIN, HIGH); delay(200);
        digitalWrite(LED_PIN, LOW);  delay(200);
      }
    }
  }
}
```

REGISTERING A CARD UID

1. Run Appendix 3 (basic read) code first. Scan your card and note the 10-character UID.
2. In the access control code, add that UID to the allowedIDs[] array.
3. Re-upload the code.

PROCEDURE

1. Connect EM18 as described. Connect LED with 470Ω to GPIO2.
2. Register at least one card UID in the allowedIDs[] array.
3. Upload the code. Open Serial Monitor at 115200 baud.
4. Scan a registered card — LED stays ON for 1 second; Serial shows "ACCESS GRANTED".
5. Scan an unregistered card — LED blinks 3 times; Serial shows "ACCESS DENIED".
6. Record UIDs and responses in the observation table.

RESULT

RESULT

An RFID-based access control system was implemented on ESP32 using the EM18 reader on Serial2 (GPIO16). XOR checksum verification confirmed data integrity. Pre-registered card UIDs triggered LED ON for 1 second (ACCESS GRANTED), while unregistered UIDs caused 3 LED blinks (ACCESS DENIED), demonstrating a functional access control system.

APPENDIX 5 — ACS712-20A CURRENT SENSOR WITH ESP32

AIM

To interface the ACS712-20A Hall-effect current sensor with the ESP32 ADC using a 10kΩ/20kΩ voltage divider and display the measured current (in amperes) on the Serial Monitor.

COMPONENTS REQUIRED

Sl.	Component	Specification	Qty	Purpose
1	Microcontroller	ESP32 Dev Board	1	ADC measurement
2	Current Sensor	ACS712-20A Module (±20A)	1	Measure current (Hall effect)
3	Resistor	10 kΩ, 1/4 W	1	Voltage divider upper
4	Resistor	20 kΩ, 1/4 W	1	Voltage divider lower
5	Test Load	Resistor/bulb (known current)	1	Current source for testing

CIRCUIT DIAGRAM

```
ACS712-20A                      ESP32
VCC  → 5V (VIN)
GND  → GND
SOUT → [10kΩ] → node → GPIO34 (ADC)
                 node → [20kΩ] → GND

Max SOUT = 4.5V → V_node = 4.5 × 20/30 = 3.0V (safe)

Current path (series with load):
Supply (+) → IP+ ———————————— IP- → Load → Supply(−)
```

THEORY

Parameter	Value / Formula
Sensitivity (ACS712-20A)	100 mV/A
Output at 0A	2.500 V (half of 5V supply)
Divider ratio k	20/(10+20) = 0.6667
ADC voltage → SOUT voltage	V_out = V_adc / 0.6667
SOUT → Current	I (A) = (V_out − 2.500) / 0.100
Averaging	500 ADC samples averaged to reduce noise

PROGRAM CODE (ESP32 — Arduino IDE)

```
#define CURRENT_PIN 34
#define SAMPLES     500
#define DIVIDER_K   0.6667   // 20k/(10k+20k)
#define SENSITIVITY 0.100    // ACS712-20A: 100 mV/A
#define V_ZERO      2.500    // output at 0A

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);
  analogSetAttenuation(ADC_11db);
  Serial.println("ACS712-20A Current Sensor");
}

void loop() {
  long sum = 0;
  for (int i = 0; i < SAMPLES; i++) {
    sum += analogRead(CURRENT_PIN);
    delayMicroseconds(100);
  }
  float adc_avg = (float)sum / SAMPLES;
  float V_adc   = (adc_avg / 4095.0) * 3.3;
  float V_out   = V_adc / DIVIDER_K;
  float current = (V_out - V_ZERO) / SENSITIVITY;

  Serial.print("ADC avg: "); Serial.print(adc_avg, 1);
  Serial.print("  V_out: "); Serial.print(V_out, 3);
  Serial.print(" V  Current: "); Serial.print(current, 3);
  Serial.println(" A");
  delay(1000);
}
```

OBSERVATION TABLE

Sl.	Load	ADC Avg	V_out (V)	Measured I (A)	Actual I (A)
1					
2					
3					
4					

PROCEDURE

1. Build the 10kΩ/20kΩ voltage divider: 10kΩ from SOUT to node; 20kΩ from node to GND; node to GPIO34.
2. Connect ACS712 VCC to 5V (VIN) and GND to GND.
3. Connect the test load in series through IP+ and IP− of the ACS712.
4. Upload the code. Open Serial Monitor at 115200 baud.
5. Power the load circuit and note the displayed current.

6. Verify with a known load (e.g., a resistor where I = V/R is known).

RESULT

RESULT

The ACS712-20A current sensor was successfully interfaced with the ESP32. The SOUT output was level-shifted from 5V to 3.3V using a 10kΩ/20kΩ voltage divider connected to GPIO34. The code averaged 500 ADC samples, converted to actual voltage, reversed the divider, and calculated current using the 100mV/A sensitivity. Measured current values matched the expected values for known test loads.

APPENDIX 6 — NEO-6M GPS MODULE WITH ESP32

AIM

To interface the u-blox NEO-6M GPS module with the ESP32 using UART2 and the TinyGPS++ library to read and display GPS location data (latitude, longitude, altitude, satellites, HDOP) on the Serial Monitor.

COMPONENTS REQUIRED

Sl.	Component	Specification	Qty	Purpose
1	Microcontroller	ESP32 Dev Board	1	Parse NMEA data
2	GPS Module	u-blox NEO-6M (3.3V TTL)	1	GPS receiver
3	GPS Antenna	Ceramic patch antenna (included)	1	Signal reception
4	Jumper Wires	Male-to-Female	4	Connections

CIRCUIT DIAGRAM

```
NEO-6M Module      ESP32
VCC  → 3.3V
GND  → GND
TX   → GPIO16 (RX2)   (no level shifting needed — 3.3V TTL)
RX   → GPIO17 (TX2)   (optional — only needed for config)

No voltage divider required — NEO-6M is 3.3V TTL, same as ESP32.
Place GPS module near a window or outdoors for satellite fix.
```

THEORY

NMEA Sentences:

Sentence	Contains	Key Fields
$GPGGA	Fix data	Latitude, Longitude, Altitude, Satellites, HDOP
$GPRMC	Recommended minimum	Lat, Lng, Speed, Date, Time
$GPGSV	Satellites in view	Satellite IDs, elevation, azimuth, SNR

TinyGPS++ library parses these NMEA sentences automatically. HDOP (Horizontal Dilution of Precision) indicates fix quality — HDOP < 2.0 is good; < 1.0 is excellent.

LIBRARY INSTALLATION

Arduino IDE → Sketch → Manage Libraries → search "TinyGPS++" → install by Mikal Hart.

PROGRAM CODE (ESP32 — Arduino IDE)

```
#include <TinyGPS++.h>
#include <HardwareSerial.h>

#define GPS_RX 16   // ESP32 GPIO16 ← GPS TX
#define GPS_TX 17   // ESP32 GPIO17 → GPS RX (optional)

TinyGPSPlus gps;
HardwareSerial gpsSerial(2);   // UART2

void setup() {
  Serial.begin(115200);
  gpsSerial.begin(9600, SERIAL_8N1, GPS_RX, GPS_TX);
  Serial.println("NEO-6M GPS — Waiting for fix...");
}

void loop() {
  while (gpsSerial.available())
    gps.encode(gpsSerial.read());

  if (gps.location.isUpdated()) {
    Serial.print("Lat: ");  Serial.print(gps.location.lat(), 6);
    Serial.print("  Lng: "); Serial.print(gps.location.lng(), 6);
    Serial.print("  Alt: "); Serial.print(gps.altitude.meters(), 1);
    Serial.print(" m  Sat: "); Serial.print(gps.satellites.value());
    Serial.print("  HDOP: "); Serial.println(gps.hdop.hdop(), 2);
  }
}
```

OBSERVATION TABLE

Reading	Latitude	Longitude	Altitude (m)	Satellites	HDOP
1					
2					
3					

PROCEDURE

1. Connect NEO-6M: VCC → 3.3V, GND → GND, TX → GPIO16, RX → GPIO17.
2. Install TinyGPS++ library in Arduino IDE.
3. Upload the code. Open Serial Monitor at 115200 baud.
4. Place the GPS module near an open window or take it outdoors for satellite reception.
5. Wait for fix (may take 1–5 minutes on first use — LED on module blinks when locked).
6. Record latitude, longitude, altitude, satellite count, and HDOP in the observation table.
7. Verify location against Google Maps using the displayed coordinates.

RESULT

RESULT

The NEO-6M GPS module was successfully interfaced with the ESP32 using UART2 (GPIO16/RX2) with direct 3.3V TTL connection — no voltage divider required. The TinyGPS++ library parsed NMEA sentences and provided latitude, longitude, altitude, satellite count, and HDOP. GPS coordinates were verified against Google Maps and found to be accurate.

APPENDIX 7 — EVENT DURATION TIMER USING PUSH BUTTON & ESP32

AIM

To measure the time duration between two push button press events using the ESP32 millis() function — the first press starts the timer and the second press stops it — and display the elapsed time in seconds (with millisecond precision) on the Serial Monitor.

COMPONENTS REQUIRED

Sl.	Component	Specification	Qty	Purpose
1	Microcontroller	ESP32 Dev Board	1	Timer & serial output
2	Push Button	Momentary tactile switch	1	Start / stop event
3	Breadboard + Jumpers	—	1 set	Connections
4	USB Cable	Micro USB / Type-C	1	Power & Serial Monitor

CIRCUIT DIAGRAM

```
Push Button    ESP32
Terminal 1 → GPIO4 (INPUT_PULLUP)
Terminal 2 → GND

No external resistor needed – ESP32 internal pull-up used.
Resting state: GPIO4 = HIGH (3.3V via internal pull-up)
Pressed state: GPIO4 = LOW  (shorted to GND)
```

THEORY

millis() returns the number of milliseconds elapsed since the ESP32 was powered on. It runs from a hardware timer continuously in the background.

Step	Operation
1st press	startTime = millis() — record start timestamp
2nd press	elapsed = millis() − startTime — duration in ms
Convert	seconds = elapsed / 1000.0 (float for decimal places)
Display	Serial.print(seconds, 3) — 3 decimal places

Software debounce: after LOW detected, wait 50ms and re-read. If still LOW → confirmed press. Wait for button release + 50ms delay prevents repeat triggering.

PROGRAM CODE (ESP32 — Arduino IDE)

```
#define BTN_PIN  4      // push button — other terminal to GND

bool          timing    = false;
unsigned long startTime = 0;

void setup() {
  Serial.begin(115200);
  pinMode(BTN_PIN, INPUT_PULLUP);
  Serial.println("Event Duration Timer Ready");
  Serial.println("Press button to START...");
}

void loop() {
  if (digitalRead(BTN_PIN) == LOW) {
    delay(50);
    if (digitalRead(BTN_PIN) == LOW) {

      if (!timing) {
        startTime = millis();
        timing    = true;
        Serial.println("Timer STARTED.");
        Serial.println("Press again to STOP...");
      } else {
        unsigned long elapsed = millis() - startTime;
        timing = false;
        Serial.println("Timer STOPPED.");
        Serial.print("Duration : ");
        Serial.print(elapsed / 1000.0, 3);
        Serial.println(" seconds");
        Serial.print("           ");
        Serial.print(elapsed);
        Serial.println(" ms");
        Serial.println("————————————————————————————");
        Serial.println("Press button to START again...");
      }

      while (digitalRead(BTN_PIN) == LOW);
      delay(50);
    }
  }
}
```

EXPECTED SERIAL OUTPUT

```
Event Duration Timer Ready
Press button to START...
Timer STARTED.
Press again to STOP...
Timer STOPPED.
Duration : 4.382 seconds
           4382 ms
————————————————————————————
```

```
Press button to START again...
```

OBSERVATION TABLE

Trial	Event	Duration (seconds)	Duration (ms)
1	Button pair 1		
2	Button pair 2		
3	Button pair 3		
4	Button pair 4		

PROCEDURE

1. Connect one terminal of the push button to GPIO4 and the other to GND. No external resistor needed.
2. Upload the code. Open Serial Monitor at 115200 baud.
3. Press the button once — Serial Monitor shows "Timer STARTED."
4. Wait 5 seconds and press the button again.
5. Observe the duration in seconds and milliseconds.
6. Record start and stop times and computed duration in the observation table.
7. Repeat for 3 more trials with different intervals.

RESULT

RESULT

The event duration timer was successfully implemented using a push button on GPIO4 (INPUT_PULLUP) and the ESP32 millis() function. The first press recorded the start timestamp; the second press computed the elapsed duration. Results were displayed with 3-decimal-place second precision (millisecond resolution) on the Serial Monitor. Software debouncing prevented false triggering.

APPENDIX 8 — ENERGY UNIT CALCULATOR (ESP32 + ACS712-20A)

AIM

To calculate electrical energy consumed by a load during a measured time interval using the ACS712-20A current sensor and ESP32. A push button starts and stops measurement. Supply voltage is assumed as 230 V AC with unity power factor.

COMPONENTS REQUIRED

Sl.	Component	Specification	Qty	Purpose
1	Microcontroller	ESP32 Dev Board	1	ADC, timer, processing
2	Current Sensor	ACS712-20A Module	1	Measure load current
3	Push Button	Momentary tactile switch	1	Start / stop measurement
4	Resistor	10 kΩ + 20 kΩ, 1/4 W	1 each	Voltage divider for SOUT
5	AC Load	Resistive (bulb/heater)	1	Load under measurement

CIRCUIT DIAGRAM

```
ACS712 VCC  → 5V (VIN)
ACS712 GND  → GND
ACS712 SOUT → [10kΩ] → node → GPIO34
                node → [20kΩ] → GND
Push button → GPIO4 (INPUT_PULLUP) | other terminal → GND

AC Load circuit (230V mains — handle with care):
230V Line → ACS712 IP+ ── [Load] ── IP− → 230V Neutral
```

WARNING: IP+/IP− carry 230V mains. ACS712 provides 2.1kV galvanic isolation between the mains side and signal side.

THEORY

Parameter	Formula / Value
Supply Voltage	230 V AC (assumed)
Power Factor	1.0 (unity — resistive load)
Current from sensor	I (A) = (V_out − 2.5) / 0.1
Power	P (W) = 230 × I × 1.0
Energy per interval	dE (Wh) = P × (dt / 3600000) [dt in ms]
Total Energy	E_total = Σ dE (sum over all samples)
Units consumed	kWh = E_total / 1000

PROGRAM CODE (ESP32 — Arduino IDE)

```
#define BTN_PIN      4
#define CURRENT_PIN  34
#define V_SUPPLY     230.0
#define POWER_FACTOR 1.0
#define SENSITIVITY  0.100    // ACS712-20A: 100 mV/A
#define V_ZERO       2.500
#define DIVIDER_K    0.6667
#define SAMPLES      150
#define SAMPLE_MS    500

bool          measuring   = false;
unsigned long startTime   = 0;
unsigned long lastSample  = 0;
float         energyWh    = 0.0;
float         peakCurrent = 0.0;

float readCurrent() {
  long sum = 0;
  for (int i = 0; i < SAMPLES; i++) {
    sum += analogRead(CURRENT_PIN);
    delayMicroseconds(100);
  }
  float V_adc   = ((float)sum / SAMPLES / 4095.0) * 3.3;
  float V_out   = V_adc / DIVIDER_K;
  return abs((V_out - V_ZERO) / SENSITIVITY);
}

void setup() {
  Serial.begin(115200);
  pinMode(BTN_PIN, INPUT_PULLUP);
  analogReadResolution(12);
  analogSetAttenuation(ADC_11db);
  Serial.println("Energy Calculator Ready — Press button to START");
}

void loop() {
  if (digitalRead(BTN_PIN) == LOW) {
    delay(50);
    if (digitalRead(BTN_PIN) == LOW) {
      if (!measuring) {
        measuring = true; startTime = millis();
        lastSample = millis(); energyWh = 0; peakCurrent = 0;
        Serial.println("\nMeasurement STARTED. Current | Power | Energy");
      } else {
        measuring = false;
        unsigned long elapsed = millis() - startTime;
        Serial.println("\n===== ENERGY REPORT =====");
        Serial.print("Duration   : "); Serial.print(elapsed/1000.0,2);
Serial.println(" s");
        Serial.print("Peak I     : "); Serial.print(peakCurrent,3);
Serial.println(" A");
```

```
        Serial.print("Energy     : "); Serial.print(energyWh,4);
Serial.println(" Wh");
        Serial.print("Units (kWh): "); Serial.println(energyWh/1000.0,7);
        Serial.println("========================");
      }
      while (digitalRead(BTN_PIN) == LOW); delay(50);
    }
  }
  if (measuring && (millis() - lastSample >= SAMPLE_MS)) {
    unsigned long dt = millis() - lastSample; lastSample = millis();
    float I = readCurrent();
    float P = V_SUPPLY * I * POWER_FACTOR;
    energyWh += P * (dt / 3600000.0);
    if (I > peakCurrent) peakCurrent = I;
    Serial.print(I,3); Serial.print(" A | ");
    Serial.print(P,1); Serial.print(" W | ");
    Serial.print(energyWh,5); Serial.println(" Wh");
  }
}
```

OBSERVATION TABLE

Trial	Load	Duration (s)	Peak I (A)	Energy (Wh)	Units (kWh)
1					
2					
3					

PROCEDURE

1. Connect ACS712 SOUT through 10kΩ/20kΩ divider to GPIO34. Connect button to GPIO4 → GND.
2. Connect the AC load in series through ACS712 IP+ and IP−.
3. Upload the code. Open Serial Monitor at 115200 baud.
4. Switch ON the AC load.
5. Press button once — current, power, and running energy appear every 500 ms.
6. Press button again to stop — energy report is displayed.
7. Record values in the observation table.

RESULT

RESULT

The energy unit calculator was successfully implemented. The ACS712-20A measured current via a 10kΩ/20kΩ divider on GPIO34. A push button on GPIO4 started and stopped measurement. Energy was integrated as dE = P × dt every 500ms and accumulated. The final report showed total duration, peak current, energy in Wh, and units in kWh.

APPENDIX 9 — DHT11/DHT22 SENSOR WITH I2C LCD AND BLYNK IoT (ESP32)

AIM

To interface the DHT11/DHT22 temperature and humidity sensor with the ESP32, display readings on a 16×2 I2C LCD, and simultaneously send data to the Blynk IoT cloud platform for real-time remote monitoring.

COMPONENTS REQUIRED

Sl.	Component	Specification	Qty	Purpose
1	Microcontroller	ESP32 Dev Board	1	WiFi, processing, ADC
2	Temp & Humidity Sensor	DHT11 or DHT22 module	1	Measure temp & humidity
3	I2C LCD Display	16×2 with PCF8574 backpack	1	Local display
4	Resistor	10 kΩ, 1/4 W	1	Pull-up for DHT DATA pin
5	Smartphone	Blynk app installed	1	Remote monitoring

CIRCUIT DIAGRAM

```
DHT11/22           ESP32
VCC  → 3.3V
GND  → GND
DATA → GPIO4  +  [10kΩ pull-up from DATA to 3.3V]

I2C LCD (PCF8574 backpack)
VCC  → VIN (5V)
GND  → GND
SDA  → GPIO21
SCL  → GPIO22

If LCD stays blank: run I2C scanner to check address (0x27 or 0x3F).
```

THEORY

Parameter	DHT11	DHT22 (AM2302)
Temp Range	0–50 °C	-40–80 °C
Temp Accuracy	±2 °C	±0.5 °C
Humidity Range	20–80% RH	0–100% RH
Humidity Accuracy	±5% RH	±2–5% RH
Sampling Rate	Max 1/second	Max 1 per 2 seconds

Blynk Virtual Pins: V0 = Temperature (°C), V1 = Humidity (%). The 2000ms timer interval satisfies both DHT11 (min 1s) and DHT22 (min 2s) requirements.

LIBRARY INSTALLATION

1. DHT sensor library — search "DHT sensor library" → install by Adafruit (also install Adafruit Unified Sensor when prompted).
2. LiquidCrystal_I2C — search "LiquidCrystal I2C" → install by Frank de Brabander.
3. Blynk — search "Blynk" → install by Blynk.

PROGRAM CODE (ESP32 — Arduino IDE)

```
#define BLYNK_TEMPLATE_ID   "TMPLxxxxxxxxx"
#define BLYNK_TEMPLATE_NAME "DHT Monitor"
#define BLYNK_AUTH_TOKEN    "YourAuthTokenHere"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <DHT.h>
#include <LiquidCrystal_I2C.h>

#define DHTPIN   4
#define DHTTYPE  DHT11      // change to DHT22 if needed

char ssid[] = "Your_WiFi_SSID";
char pass[] = "Your_WiFi_Password";

DHT               dht(DHTPIN, DHTTYPE);
LiquidCrystal_I2C lcd(0x27, 16, 2);
BlynkTimer        timer;

void sendSensorData() {
  float humidity    = dht.readHumidity();
  float temperature = dht.readTemperature();

  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("DHT read failed!");
    return;
  }
  lcd.setCursor(0, 0);
  lcd.print("Temp: "); lcd.print(temperature, 1); lcd.print(" C   ");
  lcd.setCursor(0, 1);
  lcd.print("Hum:  "); lcd.print(humidity, 1);    lcd.print(" %   ");

  Blynk.virtualWrite(V0, temperature);
  Blynk.virtualWrite(V1, humidity);

  Serial.print("Temp: "); Serial.print(temperature,1);
  Serial.print(" C | Hum: "); Serial.print(humidity,1); Serial.println(" %");
}
```

```
void setup() {
  Serial.begin(115200);
  lcd.init(); lcd.backlight();
  lcd.setCursor(0,0); lcd.print("DHT IoT Monitor");
  lcd.setCursor(0,1); lcd.print("Connecting...");
  dht.begin();
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
  lcd.clear();
  timer.setInterval(2000L, sendSensorData);
}

void loop() {
  Blynk.run();
  timer.run();
}
```

EXPECTED LCD DISPLAY

```
+————————————————+
| Temp: 28.5 C   |
| Hum:  62.3 %   |
+————————————————+
```

OBSERVATION TABLE

Sl.	Time	Temp LCD (°C)	Hum LCD (%)	Temp Blynk	Hum Blynk
1					
2					
3					
4					

PROCEDURE

1. Connect DHT DATA to GPIO4 with a 10kΩ pull-up from DATA to 3.3V.
2. Connect I2C LCD: SDA → GPIO21, SCL → GPIO22, VCC → VIN, GND → GND.
3. Install Adafruit DHT, LiquidCrystal_I2C, and Blynk libraries.
4. Set up Blynk template with V0 (Temp, 0–60°C) and V1 (Humidity, 0–100%) datastreams.
5. Update BLYNK_AUTH_TOKEN, ssid, and pass in the code. Upload.
6. Open Serial Monitor at 115200 baud — confirm WiFi connected and readings appear.
7. Check LCD — Line 1: temperature, Line 2: humidity, updating every 2 seconds.
8. Open Blynk app — verify V0 and V1 widgets receive live data.
9. Record readings from LCD and Blynk in the observation table at 4 different times.

RESULT

RESULT

The DHT11/DHT22 sensor was successfully interfaced with the ESP32. Temperature and humidity readings were displayed on the 16×2 I2C LCD (GPIO21/SDA, GPIO22/SCL) and simultaneously published to Blynk IoT cloud via WiFi using virtual pins V0 and V1. LCD and Blynk readings were consistent, confirming correct sensor reading, local display, and cloud connectivity.

APPENDIX 10 — TIPPING BUCKET RAIN GAUGE WITH ESP32 AND I2C OLED DISPLAY

AIM

To interface a tipping bucket rain gauge (reed switch output) with the ESP32 using an external interrupt to count bucket tips, calculate total rainfall and rainfall rate, and display the measurements on a 128×64 I2C OLED display (SSD1306).

COMPONENTS REQUIRED

Sl.	Component	Specification	Qty	Purpose
1	Microcontroller	ESP32 Dev Board	1	Interrupt counting, display
2	Tipping Bucket Rain Gauge	Reed switch output, 0.2 mm/tip	1	Rainfall measurement
3	OLED Display	SSD1306, 128×64, I2C (0x3C)	1	Display rainfall data
4	Resistor (optional)	10 kΩ pull-up (if no module pull-up)	1	Reed switch pull-up (ESP32 internal used)
5	Jumper Wires	Male-to-Female/Male	1 set	Connections
6	USB Cable	Micro USB / Type-C	1	Power & programming

WORKING PRINCIPLE OF TIPPING BUCKET RAIN GAUGE

A tipping bucket rain gauge collects rain water in a funnel which drains into a small see-saw bucket. When the bucket fills with a precise volume of water (equivalent to 0.2 mm of rain over the funnel area), it tips — emptying itself and bringing the other side into position. Each tip activates a reed switch (a magnetically operated switch) momentarily. Counting these switch closures gives the total rainfall.

Parameter	Description
Reed Switch Output	Momentary short circuit (normally open) — each tip closes the switch briefly
Typical Sensitivity	0.2 mm or 0.5 mm per tip — check the gauge datasheet or calibration label
Pulse Duration	~10–100 ms per tip (varies with tip speed and magnet strength)
Contact Bounce	Reed switches bounce — must debounce (50 ms delay in ISR)
ESP32 Input	GPIO configured as INPUT_PULLUP — switch pulls pin to GND on each tip
Interrupt Type	FALLING edge — triggered when reed switch closes (HIGH → LOW)

CIRCUIT DIAGRAM

```
    Tipping Bucket                ESP32
```

```
  Reed Switch
  Terminal 1 ──────────────────── GPIO4 (INPUT_PULLUP)
  Terminal 2 ──────────────────── GND

  I2C OLED SSD1306 (128×64)
  VCC ──────────────────────────  3.3V
  GND ──────────────────────────  GND
  SDA ──────────────────────────  GPIO21
  SCL ──────────────────────────  GPIO22

  Signal behaviour:
  Resting (no rain)  : GPIO4 = HIGH (internal pull-up to 3.3V)
  Each bucket tip    : GPIO4 = LOW  (reed switch pulls to GND) → FALLING edge → ISR
```

Figure A10.1: Tipping bucket reed switch → GPIO4; OLED SSD1306 → GPIO21/GPIO22

Component Pin	Connects To	Notes
Reed switch terminal 1	GPIO4	INPUT_PULLUP — FALLING interrupt attached
Reed switch terminal 2	GND	Completes circuit when switch closes
OLED VCC	3.3V	SSD1306 runs on 3.3V (some modules have onboard regulator for 5V too)
OLED GND	GND	Common ground
OLED SDA	GPIO21	ESP32 default I2C data
OLED SCL	GPIO22	ESP32 default I2C clock

THEORY

Rainfall Calculation

Parameter	Formula
Total Rainfall	Total (mm) = tipCount × MM_PER_TIP
Rainfall Rate (per 60s window)	Rate (mm/hr) = (tipsInPeriod × MM_PER_TIP) × 60
Live Rate (extrapolated)	Rate (mm/hr) = (tipsInPeriod × MM_PER_TIP) × (3600000 / elapsed_ms)
MM_PER_TIP	0.2 mm (standard) — check gauge calibration label and adjust if different

ESP32 External Interrupt

attachInterrupt(digitalPinToInterrupt(REED_PIN), onTip, FALLING) registers the onTip() function to execute automatically each time GPIO4 transitions from HIGH to LOW. The ISR is marked IRAM_ATTR to store it in IRAM (Internal RAM) so it executes even during flash operations. Since ISRs must be fast, only the counter is incremented and a debounce timestamp is checked — no Serial prints or delays inside the ISR.

SSD1306 OLED Display

Property	Value
Resolution	128 × 64 pixels
Interface	I2C (default address 0x3C; some modules use 0x3D)
Library	Adafruit SSD1306 + Adafruit GFX
Text sizes	setTextSize(1) = 6×8px per char; setTextSize(2) = 12×16px
Drawing	drawLine(), drawRect(), fillRect(), drawCircle() — from Adafruit GFX

LIBRARY INSTALLATION (Arduino IDE)

1. Open Sketch → Include Library → Manage Libraries.
2. Search "Adafruit SSD1306" → install. Also install "Adafruit GFX Library" when prompted.
3. Both libraries are required — SSD1306 uses GFX for drawing primitives.

PROGRAM CODE (ESP32 — Arduino IDE)

```
/* ============================================================
   Appendix 10 – Tipping Bucket Rain Gauge + I2C OLED (ESP32)
   Reed switch: GPIO4 (INPUT_PULLUP) – other terminal GND
   OLED SSD1306 128x64: SDA→GPIO21, SCL→GPIO22, VCC→3.3V
   Each tip = 0.2 mm rainfall (adjust MM_PER_TIP for your gauge)
   Interrupt on FALLING edge counts every tip
   ============================================================ */

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_W   128
#define SCREEN_H    64
#define OLED_ADDR  0x3C    // common SSD1306 I2C address

#define REED_PIN    4      // reed switch – INPUT_PULLUP
#define MM_PER_TIP  0.2f   // mm of rain per bucket tip (calibrate)

Adafruit_SSD1306 display(SCREEN_W, SCREEN_H, &Wire, -1);

volatile uint32_t tipCount    = 0;   // incremented in ISR
uint32_t          lastTipTime = 0;   // for debounce
uint32_t          periodStart = 0;   // start of 1-min rate window
uint32_t          tipsInPeriod = 0;  // tips in current minute

// ── Interrupt Service Routine – called on each reed switch close ──
void IRAM_ATTR onTip() {
  uint32_t now = millis();
  if (now - lastTipTime > 50) {   // 50 ms debounce
    tipCount++;
    tipsInPeriod++;
```

```
    lastTipTime = now;
  }
}

void updateDisplay(float totalMm, float rateMmHr) {
  display.clearDisplay();

  // ── Title bar ──
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(20, 0);
  display.println("RAIN GAUGE");
  display.drawLine(0, 10, 127, 10, SSD1306_WHITE);

  // ── Total rainfall ──
  display.setCursor(0, 15);
  display.setTextSize(1);
  display.print("Total Rain:");
  display.setCursor(0, 25);
  display.setTextSize(2);
  display.print(totalMm, 1);
  display.print(" mm");

  // ── Rate ──
  display.setCursor(0, 46);
  display.setTextSize(1);
  display.print("Rate: ");
  display.print(rateMmHr, 1);
  display.print(" mm/hr");

  // ── Tip count ──
  display.setCursor(0, 56);
  display.print("Tips: ");
  display.print(tipCount);

  display.display();
}

void setup() {
  Serial.begin(115200);

  // OLED init
  if (!display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
    Serial.println("SSD1306 not found — check wiring/address");
    while (true);
  }
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(10, 25);
  display.println("Rain Gauge Ready");
  display.display();
```

```
  delay(1500);

  // Reed switch interrupt
  pinMode(REED_PIN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(REED_PIN), onTip, FALLING);

  periodStart = millis();
  Serial.println("Tipping Bucket Rain Gauge Started");
  Serial.println("Tips | Total(mm) | Rate(mm/hr)");
}

void loop() {
  float totalMm  = tipCount * MM_PER_TIP;

  // ── Compute rainfall rate every 60 seconds ──
  uint32_t elapsed = millis() - periodStart;
  float rateMmHr = 0.0;
  if (elapsed >= 60000) {
    rateMmHr   = (tipsInPeriod * MM_PER_TIP) * 60.0;  // mm/hr
    tipsInPeriod = 0;
    periodStart  = millis();
  } else {
    // Live estimate: extrapolate current period to full hour
    if (elapsed > 0)
      rateMmHr = (tipsInPeriod * MM_PER_TIP) * (3600000.0 / elapsed);
  }

  updateDisplay(totalMm, rateMmHr);

  Serial.print(tipCount);
  Serial.print("  | ");
  Serial.print(totalMm, 2);
  Serial.print(" mm  | ");
  Serial.print(rateMmHr, 1);
  Serial.println(" mm/hr");

  delay(1000);   // update display every second
}
```

EXPECTED OLED DISPLAY

```
+──────────────────────────────────+
|       RAIN GAUGE                 |
|──────────────────────────────────|
| Total Rain:                      |
|  2.0 mm                          |
|                                  |
| Rate: 12.0 mm/hr                 |
| Tips: 10                         |
+──────────────────────────────────+
```

Figure A10.2: SSD1306 OLED display layout — title bar, total rainfall (large), rate and tip count below

EXPECTED SERIAL MONITOR OUTPUT (115200 baud)

```
Tipping Bucket Rain Gauge Started
Tips | Total(mm) | Rate(mm/hr)
0  | 0.00 mm  | 0.0 mm/hr
1  | 0.20 mm  | 12.0 mm/hr
2  | 0.40 mm  | 12.0 mm/hr
5  | 1.00 mm  | 6.0 mm/hr
10 | 2.00 mm  | 4.8 mm/hr
```

OBSERVATION TABLE

Time (min)	Tip Count	Total Rainfall (mm)	Rate (mm/hr)	Remarks
0				
1				
2				
3				
4				
5				

Table A10.1: Record tip count, total rainfall, and rate at 1-minute intervals

CALIBRATION NOTE

To verify MM_PER_TIP for your specific gauge:

1. Pour a known volume of water (e.g., 50 mL) slowly into the funnel.
2. Count the number of tips displayed.
3. Calculate: MM_PER_TIP = (Volume_mL / Funnel_Area_cm²) / tipCount
4. Funnel area (cm²) = π × r², where r = radius of the funnel opening in cm.
5. Update MM_PER_TIP in the code with the calibrated value and re-upload.

For a standard WS-3000 or Davis tipping bucket gauge, MM_PER_TIP = 0.2 mm. For larger gauges it may be 0.5 mm or 1.0 mm.

PROCEDURE

1. Connect the tipping bucket reed switch: Terminal 1 → GPIO4, Terminal 2 → GND.
2. Connect the OLED SSD1306: VCC → 3.3V, GND → GND, SDA → GPIO21, SCL → GPIO22.
3. Install Adafruit SSD1306 and Adafruit GFX libraries in Arduino IDE.
4. Verify the OLED I2C address (typically 0x3C). If OLED does not start, run an I2C scanner and update OLED_ADDR.
5. Check the gauge datasheet for MM_PER_TIP. Update the value in the code if needed.
6. Upload the code. Open Serial Monitor at 115200 baud.
7. Observe OLED — should show "Rain Gauge Ready", then switch to the measurement screen.
8. Simulate rain by manually tipping the bucket by hand or pouring a small amount of water.

9. Observe tip count, total rainfall, and rate update on both OLED and Serial Monitor.
10. Record values at 1-minute intervals in the observation table.
11. Verify: Total (mm) = tip count × 0.2. Compare with observation.

RESULT

RESULT

The tipping bucket rain gauge was successfully interfaced with the ESP32. Each bucket tip closed the reed switch, generating a FALLING edge on GPIO4 which triggered the IRAM_ATTR ISR via attachInterrupt(). A 50ms software debounce prevented false counts. Total rainfall was calculated as tipCount × 0.2 mm and displayed on the SSD1306 128×64 I2C OLED (GPIO21/SDA, GPIO22/SCL) along with the live rainfall rate in mm/hr, updated every second. Serial Monitor provided a concurrent log of all measurements.

APPENDIX 11 — IR SENSOR / PIR SENSOR AND RELAY MODULE INTERFACING WITH RASPBERRY PI

AIM

To interface an IR obstacle sensor and a PIR motion sensor with the Raspberry Pi GPIO, and control a relay module based on the sensor output using Python (RPi.GPIO library).

COMPONENTS REQUIRED

Sl.	Component	Specification	Qty	Purpose
1	Single Board Computer	Raspberry Pi (any 40-pin model)	1	Main controller
2	IR Obstacle Sensor	FC-51 / LM393 IR module	1	Detect object / obstacle
3	PIR Motion Sensor	HC-SR501	1	Detect human presence
4	Relay Module	5V SPDT with optocoupler	1	Switch load
5	LED + 470 Ω resistor	Any colour	1	Load for relay (test)
6	Jumper Wires	Female-to-Female	1 set	GPIO connections

PART A — IR OBSTACLE SENSOR WITH RELAY

Working Principle

An IR obstacle sensor module (FC-51 / LM393) has an IR LED and an IR photodiode. The IR LED continuously emits infrared light. When an object comes close, the reflected IR hits the photodiode — the LM393 comparator outputs LOW (active-LOW). When the path is clear, the output is HIGH.

Condition	IR Module OUT	GPIO17 reads	Relay action
Object / obstacle in front	LOW (0V)	GPIO.LOW	ON — load activated
Path clear	HIGH (3.3V)	GPIO.HIGH	OFF — load deactivated

Circuit — IR Sensor and Relay

```
IR Sensor Module          Raspberry Pi (40-pin)
VCC  ──────────────────── Pin 1  (3.3V)
GND  ──────────────────── Pin 6  (GND)
OUT  ──────────────────── Pin 11 (GPIO17, BCM)

Relay Module
VCC  ──────────────────── Pin 4  (5V)
GND  ──────────────────── Pin 9  (GND)
IN   ──────────────────── Pin 13 (GPIO27, BCM)
```

```
Relay contacts (test load):
5V ─── COM ─── NO ─── LED (470Ω) ─── GND
```

Figure A11.1: IR sensor → GPIO17; Relay module → GPIO27; LED as test load on relay NO contact

Pin	From	To	Notes
IR VCC	IR Sensor	RPi Pin 1 (3.3V)	FC-51 works on 3.3V and 5V
IR GND	IR Sensor	RPi Pin 6 (GND)	Common ground
IR OUT	IR Sensor	RPi Pin 11 (GPIO17)	Active-LOW — LOW when obstacle
Relay VCC	Relay module	RPi Pin 4 (5V)	Relay coil needs 5V
Relay GND	Relay module	RPi Pin 9 (GND)	Common ground
Relay IN	Relay module	RPi Pin 13 (GPIO27)	3.3V from RPi — most modules accept this

Program Code — IR Sensor + Relay (Python 3)

```
#!/usr/bin/env python3
# IR Obstacle Sensor + Relay — Raspberry Pi
# IR sensor OUT → GPIO17 (BCM)  |  Relay IN → GPIO27 (BCM)
# IR module output: LOW = obstacle detected, HIGH = clear (active-LOW)

import RPi.GPIO as GPIO
import time

IR_PIN    = 17   # IR sensor OUT pin
RELAY_PIN = 27   # Relay module IN pin

GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(IR_PIN,    GPIO.IN)
GPIO.setup(RELAY_PIN, GPIO.OUT)
GPIO.output(RELAY_PIN, GPIO.HIGH)  # relay OFF (active-LOW module)

print("IR Sensor + Relay Test — Press Ctrl+C to stop")

try:
    while True:
        if GPIO.input(IR_PIN) == GPIO.LOW:   # obstacle detected
            GPIO.output(RELAY_PIN, GPIO.LOW)  # relay ON
            print("Obstacle detected — Relay ON")
        else:                                  # path clear
            GPIO.output(RELAY_PIN, GPIO.HIGH) # relay OFF
            print("Clear           — Relay OFF")
        time.sleep(0.5)

except KeyboardInterrupt:
    print("Stopped.")
finally:
```

```
    GPIO.output(RELAY_PIN, GPIO.HIGH)  # relay OFF on exit
    GPIO.cleanup()
```

Expected Output — IR Sensor

```
IR Sensor + Relay Test — Press Ctrl+C to stop
Clear           — Relay OFF
Clear           — Relay OFF
Obstacle detected — Relay ON
Obstacle detected — Relay ON
Clear           — Relay OFF
```

PART B — PIR MOTION SENSOR WITH RELAY

Working Principle

The HC-SR501 PIR sensor detects infrared radiation emitted by warm bodies. When motion is detected, the output pin goes HIGH (3.3V). When no motion, it stays LOW. The sensitivity (range) and time delay can be adjusted using two potentiometers on the module.

Condition	PIR OUT	GPIO17 reads	Relay action
Motion / person detected	HIGH (3.3V)	GPIO.HIGH	ON — load activated
No motion	LOW (0V)	GPIO.LOW	OFF — load deactivated

HC-SR501 Adjustment

Potentiometer	Function	Direction to Increase
Sx (left)	Detection range: 3 m – 7 m	Clockwise
Tx (right)	Output pulse width: 5 s – 200 s	Clockwise
Jumper H/L	H = repeat trigger (stays HIGH while motion present) L = single pulse per detection	Set to H for continuous control

Circuit — PIR Sensor and Relay

```
HC-SR501 PIR              Raspberry Pi (40-pin)
VCC  ──────────────────── Pin 2  (5V)   ← PIR needs 5V
GND  ──────────────────── Pin 6  (GND)
OUT  ──────────────────── Pin 11 (GPIO17, BCM)

Relay Module
VCC  ──────────────────── Pin 4  (5V)
GND  ──────────────────── Pin 9  (GND)
IN   ──────────────────── Pin 13 (GPIO27, BCM)

Relay contacts (test load):
5V ─── COM ─── NO ─── LED (470Ω) ─── GND
```

Note: PIR VCC must be 5V (Pin 2 or Pin 4). PIR OUT is 3.3V and is safe to connect directly to GPIO17.

Program Code — PIR Sensor + Relay (Python 3)

```
#!/usr/bin/env python3
# PIR Motion Sensor + Relay — Raspberry Pi
# PIR OUT → GPIO17 (BCM)  |  Relay IN → GPIO27 (BCM)
# PIR output: HIGH = motion detected, LOW = no motion (active-HIGH)

import RPi.GPIO as GPIO
import time

PIR_PIN   = 17   # HC-SR501 OUT pin
RELAY_PIN = 27   # Relay module IN pin

GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(PIR_PIN,   GPIO.IN)
GPIO.setup(RELAY_PIN, GPIO.OUT)
GPIO.output(RELAY_PIN, GPIO.HIGH)  # relay OFF at start

print("Waiting 30s for PIR warm-up...")
time.sleep(30)
print("PIR Sensor + Relay Ready — Press Ctrl+C to stop")

try:
    while True:
        if GPIO.input(PIR_PIN) == GPIO.HIGH:  # motion detected
            GPIO.output(RELAY_PIN, GPIO.LOW)   # relay ON
            print("Motion detected — Relay ON")
        else:                                   # no motion
            GPIO.output(RELAY_PIN, GPIO.HIGH)  # relay OFF
            print("No motion      — Relay OFF")
        time.sleep(0.5)

except KeyboardInterrupt:
    print("Stopped.")
finally:
    GPIO.output(RELAY_PIN, GPIO.HIGH)
    GPIO.cleanup()
```

Expected Output — PIR Sensor

```
Waiting 30s for PIR warm-up...
PIR Sensor + Relay Ready — Press Ctrl+C to stop
No motion      — Relay OFF
No motion      — Relay OFF
Motion detected — Relay ON
Motion detected — Relay ON
Motion detected — Relay ON
No motion      — Relay OFF
```

RELAY MODULE — ACTIVE-LOW vs ACTIVE-HIGH

Most 5V relay modules with optocouplers are active-LOW — the relay energises when IN pin is LOW and de-energises when IN is HIGH. This is the opposite of the sensor logic.

Relay Type	Relay ON when IN =	Relay OFF when IN =	GPIO.output to turn ON
Active-LOW (most modules)	LOW (0V)	HIGH (3.3V)	GPIO.LOW
Active-HIGH	HIGH (3.3V)	LOW (0V)	GPIO.HIGH

Both codes above assume active-LOW relay module. If your relay turns ON at startup and turns OFF when triggered, swap GPIO.LOW and GPIO.HIGH for RELAY_PIN in the code.

RASPBERRY PI GPIO PIN MAP (REFERENCE)

Physical Pin	BCM GPIO	Function	Used For
Pin 1	—	3.3V Power	IR sensor VCC (3.3V)
Pin 2	—	5V Power	PIR sensor VCC (5V)
Pin 4	—	5V Power	Relay module VCC
Pin 6	—	GND	Common ground
Pin 9	—	GND	Relay GND
Pin 11	GPIO17	Digital Input	Sensor OUT (IR / PIR)
Pin 13	GPIO27	Digital Output	Relay module IN

PROCEDURE

1. Connect the IR sensor: VCC → Pin 1 (3.3V), GND → Pin 6, OUT → Pin 11 (GPIO17).
2. Connect the relay module: VCC → Pin 4 (5V), GND → Pin 9, IN → Pin 13 (GPIO27).
3. Connect the test LED with 470Ω resistor between relay NO contact and 5V / GND.
4. Open Terminal on Raspberry Pi. Create the script: nano ir_relay.py
5. Paste the IR sensor code. Save (Ctrl+X → Y → Enter).
6. Run: python3 ir_relay.py
7. Bring an object close to the IR sensor — LED and relay should activate.
8. Move the object away — relay should deactivate.
9. Record observations in the table.
10. Stop the script (Ctrl+C). Create a new file: nano pir_relay.py
11. Paste the PIR sensor code. Save and run: python3 pir_relay.py
12. Wait 30 seconds for the PIR to warm up.
13. Walk in front of the PIR sensor — relay and LED activate.
14. Stand still or move away — relay deactivates after the Tx delay.
15. Compare the behaviour of the IR sensor (detects object) vs PIR sensor (detects motion/heat).

OBSERVATION TABLE

Sl.	Sensor Used	Condition	Sensor OUT	Relay State	LED
1	IR	Obstacle in front	LOW		
2	IR	Path clear	HIGH		
3	PIR	Motion detected	HIGH		
4	PIR	No motion	LOW		

RESULT

RESULT

The IR obstacle sensor (FC-51) and PIR motion sensor (HC-SR501) were successfully interfaced with the Raspberry Pi GPIO using Python 3 and the RPi.GPIO library. The IR sensor output (active-LOW) on GPIO17 and the relay module IN (active-LOW) on GPIO27 were correctly read and controlled respectively. When an obstacle was detected by the IR sensor (OUT = LOW), the relay was energised (IN = LOW) and the connected LED load turned ON. Similarly, when the PIR sensor detected motion (OUT = HIGH), the relay was energised accordingly. GPIO.cleanup() ensured safe shutdown of all pins after the script exited.

APPENDIX 12 — SENDING DATA TO FIREBASE REALTIME DATABASE (ESP32)

AIM

To send sensor data (counter, uptime, temperature, humidity) from the ESP32 to the Google Firebase Realtime Database over WiFi and verify the data in the Firebase console in real time.

COMPONENTS REQUIRED

Sl.	Component	Specification	Qty	Purpose
1	Microcontroller	ESP32 Dev Board	1	WiFi + Firebase client
2	DHT11 Sensor (Part B)	DHT11 module	1	Real sensor data to upload
3	10 kΩ Resistor (Part B)	Pull-up for DHT DATA	1	DHT signal stability
4	USB Cable	Micro USB / Type-C	1	Power & programming
5	Smartphone / PC	With internet browser	1	Firebase console verification

INTRODUCTION TO FIREBASE REALTIME DATABASE

Firebase Realtime Database is a cloud-hosted NoSQL database by Google. Data is stored as JSON and synchronised to all connected clients in real time. For IoT projects, the ESP32 sends data as key-value pairs to a database URL. The data can be viewed, queried, and used by any web or mobile application connected to the same Firebase project.

Feature	Details
Database type	NoSQL — data stored as a JSON tree
Protocol	HTTPS REST API (library handles this internally)
Free tier	1 GB storage, 10 GB/month download, 100 simultaneous connections
Latency	Real-time sync — data appears in console within 1–2 seconds
Alternatives	ThingSpeak (time-series), Adafruit IO, AWS IoT, Google Sheets via Apps Script

FIREBASE PROJECT SETUP

Step 1 — Create Firebase Project

1. Open https://console.firebase.google.com and sign in with your Google account.
2. Click "Add project" → enter a project name (e.g., ESP32-IoT-Lab) → Continue.
3. Disable Google Analytics (not needed) → Create project → Continue.

Step 2 — Enable Realtime Database

1. In the left sidebar click "Build" → "Realtime Database".
2. Click "Create Database" → choose a location (e.g., us-central1) → Next.
3. Select "Start in test mode" (allows read/write without authentication for 30 days) → Enable.
4. Note the database URL shown at the top — it looks like: your-project-default-rtdb.firebaseio.com

Step 3 — Get Database Secret Key

1. Click the gear icon (Project Settings) → Service Accounts tab.
2. Scroll down to "Database secrets" → click "Show" → copy the secret key.
3. This is your FIREBASE_AUTH value in the code.
4. (Newer Firebase projects use authentication tokens — if secret is not available, see the note below.)

Note: For newer Firebase projects, use the Legacy Token method. Go to Project Settings → General → copy the Web API Key. Then use Firebase.begin(host, apiKey) and set database rules to allow public read/write temporarily for testing.

Step 4 — Database Security Rules (Test Mode)

```
{
  "rules": {
    ".read": true,
    ".write": true
  }
}
```

These rules allow anyone to read and write. For production/real deployment, add authentication. Change rules at: Realtime Database → Rules tab.

LIBRARY INSTALLATION (Arduino IDE)

1. Open Arduino IDE → Sketch → Include Library → Manage Libraries.
2. Search "Firebase ESP32 Client" → install by Mobizt.
3. Also ensure "ArduinoJson" is installed (search "ArduinoJson" by Benoit Blanchon).
4. For Part B: also install "DHT sensor library" by Adafruit.

DATA STRUCTURE IN FIREBASE

Data is organised as a JSON tree. Each Firebase.setXxx() call writes to a path inside the database:

```
{
  "esp32": {
    "count"  : 12,
    "uptime" : 60.5,
    "status" : "online"
  },
  "dht": {
    "temperature" : 28.5,
    "humidity"    : 62.3,
    "status"      : "online"
  }
}
```

Figure A12.1: Firebase Realtime Database JSON structure after the ESP32 sends data

Function	Data Type	Example
Firebase.setInt(fbdo, path, value)	Integer	Firebase.setInt(fbdo, "/esp32/count", 5)
Firebase.setFloat(fbdo, path, value)	Float	Firebase.setFloat(fbdo, "/dht/temp", 28.5)
Firebase.setString(fbdo, path, value)	String	Firebase.setString(fbdo, "/status", "ok")
Firebase.setBool(fbdo, path, value)	Boolean	Firebase.setBool(fbdo, "/relay", true)
Firebase.getFloat(fbdo, path)	Read float	float t = fbdo.floatData()
Firebase.getString(fbdo, path)	Read string	String s = fbdo.stringData()

PART A — PROGRAM CODE: BASIC (NO SENSOR)

This code requires no sensor. It sends a counter, uptime in seconds, and a status string to Firebase every 5 seconds. Use this to verify Firebase connectivity first.

```
/* ============================================================
   Appendix 12 — Send Data to Firebase Realtime Database
   Part A: Basic (no sensor) — sends counter + uptime
   Library: FirebaseESP32 by Mobizt
            (Arduino Library Manager → search "Firebase ESP32 Client")
   ============================================================ */

#include <WiFi.h>
#include <FirebaseESP32.h>

// ── Firebase credentials ──────────────────────────────
#define FIREBASE_HOST  "your-project-default-rtdb.firebaseio.com"
#define FIREBASE_AUTH  "YourDatabaseSecretKeyHere"

// ── WiFi credentials ──────────────────────────────────
#define WIFI_SSID  "Your_WiFi_SSID"
#define WIFI_PASS  "Your_WiFi_Password"

FirebaseData fbdo;   // Firebase data object

int   count    = 0;
unsigned long sendTime = 0;

void setup() {
  Serial.begin(115200);

  // Connect to WiFi
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500); Serial.print(".");
  }
  Serial.println("\nWiFi Connected: " + WiFi.localIP().toString());
```

```
  // Connect to Firebase
  Firebase.begin(FIREBASE_HOST, FIREBASE_AUTH);
  Firebase.reconnectWiFi(true);
  Serial.println("Firebase Connected.");
}

void loop() {
  if (millis() - sendTime >= 5000) {   // send every 5 seconds
    sendTime = millis();
    count++;

    float uptime = millis() / 1000.0;

    // ── Write to Firebase paths ──────────────────────────
    if (Firebase.setInt(fbdo,    "/esp32/count",   count))
      Serial.print("count OK  ");
    else Serial.print("count FAIL: " + fbdo.errorReason() + "  ");

    if (Firebase.setFloat(fbdo,  "/esp32/uptime",  uptime))
      Serial.print("uptime OK  ");
    else Serial.print("uptime FAIL  ");

    if (Firebase.setString(fbdo, "/esp32/status",  "online"))
      Serial.println("status OK");
    else Serial.println("status FAIL");

    Serial.print("Sent → count: "); Serial.print(count);
    Serial.print("  uptime: ");     Serial.print(uptime, 1);
    Serial.println(" s");
  }
}
```

PART B — PROGRAM CODE: WITH DHT11 SENSOR

Connect DHT11 DATA to GPIO4 (10kΩ pull-up to 3.3V). This code sends real temperature and humidity to Firebase.

```
/* Part B – Send DHT11 Temperature + Humidity to Firebase */

#include <WiFi.h>
#include <FirebaseESP32.h>
#include <DHT.h>

#define FIREBASE_HOST  "your-project-default-rtdb.firebaseio.com"
#define FIREBASE_AUTH  "YourDatabaseSecretKeyHere"
#define WIFI_SSID      "Your_WiFi_SSID"
#define WIFI_PASS      "Your_WiFi_Password"

#define DHTPIN  4
#define DHTTYPE DHT11
```

```
FirebaseData fbdo;
DHT dht(DHTPIN, DHTTYPE);
unsigned long sendTime = 0;

void setup() {
  Serial.begin(115200);
  dht.begin();
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nWiFi OK");
  Firebase.begin(FIREBASE_HOST, FIREBASE_AUTH);
  Firebase.reconnectWiFi(true);
  Serial.println("Firebase OK");
}

void loop() {
  if (millis() - sendTime >= 5000) {
    sendTime = millis();

    float temp = dht.readTemperature();
    float hum  = dht.readHumidity();

    if (isnan(temp) || isnan(hum)) {
      Serial.println("DHT read failed"); return;
    }

    Firebase.setFloat(fbdo, "/dht/temperature", temp);
    Firebase.setFloat(fbdo, "/dht/humidity",    hum);
    Firebase.setString(fbdo,"/dht/status",      "online");

    Serial.print("Temp: "); Serial.print(temp);
    Serial.print(" C  Hum: "); Serial.print(hum);
    Serial.println(" %  → Firebase sent");
  }
}
```

EXPECTED SERIAL MONITOR OUTPUT (115200 baud)

```
Connecting to WiFi.....
WiFi Connected: 192.168.1.105
Firebase Connected.
count OK  uptime OK  status OK
Sent → count: 1  uptime: 5.0 s
count OK  uptime OK  status OK
Sent → count: 2  uptime: 10.0 s
count OK  uptime OK  status OK
Sent → count: 3  uptime: 15.1 s
```

VERIFYING DATA IN FIREBASE CONSOLE

1. Open https://console.firebase.google.com → your project → Realtime Database.

2. The JSON tree updates live as the ESP32 sends data — you will see the values change every 5 seconds.

3. Expand the tree: click the arrow next to the database root → esp32 → count / uptime / status.

4. Each value refreshes automatically — no need to reload the page.

```
Firebase Console → Realtime Database view:

your-project-default-rtdb
  └─ esp32
       ├─ count    : 7
       ├─ status   : "online"
       └─ uptime   : 35.1
  └─ dht
       ├─ humidity    : 62.3
       ├─ status      : "online"
       └─ temperature : 28.5
```

OBSERVATION TABLE

Sl.	Time (s)	count (Firebase)	uptime (Firebase)	Temp (°C)	Hum (%)
1	5	1			
2	10	2			
3	15	3			
4	20	4			
5	25	5			

Table A12.1: Record values observed in Firebase console and Serial Monitor at 5-second intervals

COMPARISON OF IoT DATABASE OPTIONS

Platform	Protocol	Free Tier	Best For	ESP32 Library
Firebase RTDB	HTTPS / WebSocket	1 GB, 100 conn.	Real-time dashboards	FirebaseESP32 (Mobizt)
ThingSpeak	HTTPS REST	3M msgs/year	Time-series charts	ThingSpeak library
Adafruit IO	MQTT / HTTPS	5 feeds, 30 msg/min	Simple dashboards	Adafruit MQTT library
AWS IoT Core	MQTT / HTTPS	Free tier 12 months	Production / enterprise	MQTT + PubSubClient
Google Sheets	HTTPS (Apps Script)	Unlimited (free)	Logging / reports	WiFiClientSecure

PROCEDURE

1. Create a Firebase project at console.firebase.google.com and enable Realtime Database in test mode.

2. Note the database URL (e.g., your-project-default-rtdb.firebaseio.com) and database secret key.

3. Install FirebaseESP32 by Mobizt and ArduinoJson in Arduino IDE Library Manager.

4. Open the Part A code. Replace FIREBASE_HOST, FIREBASE_AUTH, WIFI_SSID, and WIFI_PASS with your credentials.

5. Upload the code. Open Serial Monitor at 115200 baud.

6. Confirm "WiFi Connected" and "Firebase Connected" messages appear.

7. Observe "count OK uptime OK status OK" repeating every 5 seconds.

8. Open Firebase console → Realtime Database — verify values updating in real time.

9. Record values in the observation table at each 5-second interval.

10. For Part B: connect DHT11 to GPIO4, replace code with Part B version, re-upload, and verify temperature/humidity in Firebase.

11. Try modifying a value directly in the Firebase console (click on a value and edit) — the ESP32 code can read it back using Firebase.getFloat() or Firebase.getString().

RESULT

RESULT

The ESP32 was successfully connected to the Google Firebase Realtime Database over WiFi using the FirebaseESP32 library by Mobizt. Data values (counter, uptime, and status string) were written to specific JSON paths in the database using Firebase.setInt(), Firebase.setFloat(), and Firebase.setString() functions every 5 seconds. The data was verified in real time in the Firebase console, where values updated automatically without page refresh. In Part B, live DHT11 temperature and humidity readings were also uploaded and verified in the database, demonstrating a complete IoT cloud data logging system.